



**Faculdade de Design,
Tecnologia e Comunicação**
Universidade Europeia

MyKondô

Relatório de Projeto Semestral

**Elaborado no âmbito da UC Projecto de Desenvolvimento de
Software**

Estudantes:

Evandra Gomes | 20230416

José Morais | 20230315

Curso: Engenharia Informática

Orientador: Prof. Fernando Marson

Lisboa, 15 de Dezembro de 2025

Índice

Introdução.....	3
Engenharia de Software.....	4
Papel da engenharia de software no desenvolvimento do projeto.....	4
Análise de utilizadores e requisitos.....	4
Personas.....	4
User Stories.....	5
Requisitos Funcionais.....	6
Requisitos não funcionais.....	7
Diagramas relevantes para o projeto.....	8
User cases.....	8
Modelação do sistema.....	9
Arquitetura da Solução e Decisões Tecnológicas.....	11
Inteligencia Artificial.....	12
Motivação e impacto na experiência do user.....	12
Formulação do Problema.....	13
Modelo CSP e Processo de Decisão.....	13
Implementação, Testes e Métricas.....	14
Definição das Variáveis e Domínios.....	14
Modelação das restrições (Hard Constraints).....	15
Função de validação CSP.....	16
Avaliação das Soluções (Soft Constraints).....	17
Algoritmo de procura com Backtracking.....	19
Função principal.....	20
Análise Crítica, Limitações e Trabalhos Futuros.....	21
Sistemas Distribuídos.....	23
Objetivos da arquitetura.....	23
Componentes.....	24
Fluxo de Tráfego.....	25
Determinação de papéis do Banco e roteamento.....	26
Estratégias de Failover e Failback.....	26
Consistência de dados.....	26
Implementações futuras.....	29
Cibersegurança.....	30
Importância da Segurança no Contexto do Projeto.....	30
Autenticação e Autorização.....	30
Segurança das Comunicações.....	30
Proteção e Gestão de Dados.....	31
Boas Práticas de Desenvolvimento Seguro.....	31
Limitações e Evoluções Futuras.....	31
Conclusão.....	32
Reflexão Crítica e Trabalho Futuro.....	33

Introdução

A gestão de uma habitação envolve um conjunto de responsabilidades contínuas, como o acompanhamento financeiro, o acesso a informação relevante e a manutenção do espaço. No contexto dos condomínios, estas responsabilidades tornam-se mais complexas, uma vez que dependem da coordenação entre múltiplos intervenientes, da existência de regras comuns e da mediação por parte da administração. Neste cenário, a organização da informação e a eficácia da comunicação assumem um papel determinante no bom funcionamento do condomínio.

A administração e gestão de condomínios engloba um vasto conjunto de tarefas administrativas e operacionais, nomeadamente a recolha de quotizações, a gestão financeira, o tratamento de pedidos de manutenção e a comunicação regular com os moradores. Em muitos casos, estas atividades continuam a ser suportadas por métodos manuais ou por ferramentas de uso genérico que não foram concebidas especificamente para este domínio. Como consequência, surgem dados dispersos, dificuldades no acompanhamento dos processos e atrasos na tomada de decisão.

Adicionalmente, a comunicação entre moradores e administração é frequentemente realizada através de canais informais, como correio eletrónico ou aplicações de mensagens. Esta dispersão compromete a rastreabilidade dos pedidos, a centralização da informação e a transparência dos processos, reduzindo a eficácia da gestão condominial e aumentando o esforço necessário para garantir o funcionamento regular dos edifícios.

O projeto **MyKondô** surge como resposta a estes desafios, tendo como objetivo o desenvolvimento de uma plataforma digital integrada que permita modernizar e centralizar a gestão de condomínios. A solução pretende facilitar a interação entre moradores e administradores, reduzir a dependência de processos manuais e promover uma gestão mais eficiente, acessível e transparente.

Entre os objetivos específicos do projeto destacam-se a centralização da informação financeira e administrativa, a facilitação da submissão e acompanhamento de pedidos de manutenção, a simplificação dos processos de pagamento e emissão de recibos, a melhoria da comunicação através da publicação de avisos e documentação relevante, bem como o apoio à tomada de decisão da administração.

A plataforma disponibiliza interfaces adaptadas aos diferentes perfis de utilizador, permitindo que moradores e administradores acedam às funcionalidades relevantes de forma simples e organizada. A solução assenta numa arquitetura que integra aplicações cliente, serviços de backend e uma base de dados centralizada, garantindo consistência da informação e suporte à evolução futura do sistema.

Engenharia de Software

Papel da engenharia de software no desenvolvimento do projeto

A Engenharia de Software desempenhou um papel central no desenvolvimento do projeto MyKondô, forneceu-nos uma abordagem estruturada para a análise do problema, a definição da solução e implementação do sistema. Ao aplicarmos os princípios de Engenharia de Software conseguimos alinhar as necessidades dos utilizadores com as decisões técnicas, o que garante consistência, acompanhar passos e capacidade de evolução ao longo do projeto.

Desde as fases iniciais, foram adotadas práticas de análise e modelação que permitiram compreender o contexto do problema e estruturar a solução em pequenas etapas. Esta abordagem tornou-se fundamental para lidar com a complexidade do sistema e para acomodar alterações e ajustes identificados durante o desenvolvimento.

Análise de utilizadores e requisitos

A análise de utilizadores constituiu o ponto de partida para a definição das funcionalidades do sistema. Foram identificados diferentes perfis de utilizador, com necessidades e objetivos distintos, o que permitiu orientar o desenho da solução de forma centrada no utilizador.

Com base nesta análise, foram definidas **personas** representativas dos principais perfis do sistema, assim como **user stories**, utilizadas para expressar as funcionalidades do ponto de vista dos utilizadores finais. Estas user stories serviram de base para a identificação dos **requisitos funcionais e não funcionais**, assegurando que o sistema responde às necessidades reais do contexto condominial.

Personas

Nome e tipo	Descrição	Objetivo na app
Paulo Cristóvão (Morador)	Vive uma rotina profissional intensa e quer resolver assuntos do condomínio rapidamente.	Consultar e pagar as faturas de forma simples e obter recibos imediatos.
Florinda Garcia (Gestora de Condomínio)	Gere vários condomínios e necessita centralizar informações para reduzir trabalho manual e erros.	Publicar avisos, gerir pedidos e acompanhar pagamentos num único sistema.

User Stories

1.

[Como] morador, [quero] reportar um problema do condomínio, [para] informar a administração e agilizar a resolução.

Critério de aceitação: O pedido deve ser registrado com estado “Pendente” e aparecer na lista de pedidos do administrador.

2.

[Como] gestor de condomínios, [quero] visualizar o histórico de recibos, [para] confirmar facilmente quais são as faturas que já foram pagas.

Critério de aceitação: O sistema deve apresentar os recibos ordenados por data e permitir o download individual.

3.

[Como] moradora, [quero] usar diferentes métodos de pagamento [para] pagar o condomínio de forma mais conveniente.

Critério de aceitação: A app deve permitir selecionar pelo menos dois métodos distintos para refutar pagamentos, ex.: MBway e cartão.

4.

[Como] gestor de condomínios, [quero] publicar avisos e notícias [para] manter todos os moradores informados de forma centralizada.

Critério de aceitação: Os avisos devem ficar imediatamente visíveis na HomePage do utilizador.

Requisitos Funcionais

Descrevem as ações que o sistema deve permitir ou executar, correspondendo ao comportamento observável da aplicação perante os utilizadores.

Gestão de Autenticação	O sistema deve permitir que moradores e administradores façam login de forma segura
Consulta e Pagamento de Faturas	O morador deve poder visualizar faturas pendentes e efetuar pagamentos pela app.
Emissão de Recibos	Após o pagamento, o sistema deve gerar um recibo digital.
Registo de Pedidos de Manutenção	O morador deve poder submeter pedidos de manutenção através de um formulário.
Gestão de Pedidos pelo Administrador	O administrador deve poder alterar o estado dos pedidos (aprovado, rejeitado).
Publicação de Avisos	O administrador deve conseguir publicar avisos e documentos para consulta pelos moradores.
Integração do CSP	O sistema deve permitir que o módulo de IA (CSP) recomende fornecedores adequados consoante o pedido.
Histórico de Operações	O sistema deve registar todas as operações importantes (pagamentos, pedidos, alterações de estado)

Requisitos não funcionais

Referem-se às propriedades de qualidade que garantem que a solução será eficiente, segura, acessível e tecnicamente robusta.

Usabilidade	A interface deve ser intuitiva e acessível, permitindo uso confortável por utilizadores acima de 70 anos.
Performance	O sistema deve responder às ações do utilizador em menos de 2 segundos em média.
Segurança	Deve existir autenticação e autorização seguras, com encriptação de dados sensíveis.
Escalabilidade	A arquitetura deve permitir adicionar novos módulos (ex.: mais IA ou novas funcionalidades).
Disponibilidade	A aplicação deve estar disponível pelo menos 95% do tempo.
Compatibilidade	A aplicação deve funcionar em dispositivos Android (prioritário) e navegadores modernos no caso da interface web.

A definição clara destes requisitos assegura que todas as etapas seguintes: modelação, implementação e testes sejam conduzidas com consistência e alinhamento com os objetivos do MyKondô.

Diagramas relevantes para o projeto

User cases

A formalização dos requisitos identificados foi realizada por meio de casos de utilização, o que permitiu descrever de maneira estruturada as interações entre os utilizadores e o sistema. Estes casos de utilização, por sua vez, fundamentaram a elaboração dos diagramas UML, nomeadamente os diagramas de casos de uso, de sequência, de atividades e de classes.

User Case Principal: Gestão de Faturas

Ator: Morador

Descrição: O utilizador acede à aplicação móvel, consulta as faturas em atraso e realiza o pagamento através do método disponível. O sistema regista a transação e disponibiliza o recibo digital.

Pré-condições: O morador está autenticado e possui faturas associadas

Pós-condições: O pagamento é registrado e o recibo fica guardado no histórico.

User Case Secundário 1: Pedido de Manutenção

Ator: Morador

Descrição: O morador preenche um formulário com o problema identificado e submete o pedido. O administrador recebe e gere o estado do pedido.

Pré-condições: Autenticação válida.

Pós-condições: O pedido fica registrado como “Pendente”.

User Case Secundário 2: Consulta de Avisos e Documentos

Ator: Morador

Descrição: O utilizador navega até à seção “Info” para visualizar avisos publicados ou acede ao perfil para consultar documentos da sua propriedade.

Pré-condições: Autenticação válida.

Pós-condições: O utilizador visualiza a informação relevante.

Modelação do sistema

Para mostrar diagramas focamo-nos nos pagamentos que serviram como guia para a realização das outras funcionalidades:

:

User Case: Gestão de Faturas

Elemento	Descrição
Ator Principal	Morador
Objetivo	Pagar faturas pendentes e realizar o pagamento com emissão automática do recibo.
Pré Condições	O morador tem de estar autenticado e possuir faturas associadas à sua propriedade.
Fluxo Principal	- O morador acede à secção “Finanças” na app; - O sistema apresenta a lista de faturas pendentes; - O morador seleciona uma fatura e escolhe o método de pagamento; - O sistema processa o pagamento com os dados fornecidos; - O sistema confirma a transação e disponibiliza o recibo.
Fluxo Alternativo	Se o método de pagamentos for inválido ou houver erro na transação, o sistema exibe uma mensagem de erro e a fatura permanece como pendente.
Pós Condições	O pagamento é registado na BD e o recibo digital fica disponível no histórico de pagamentos.

Diagrama de use case UML

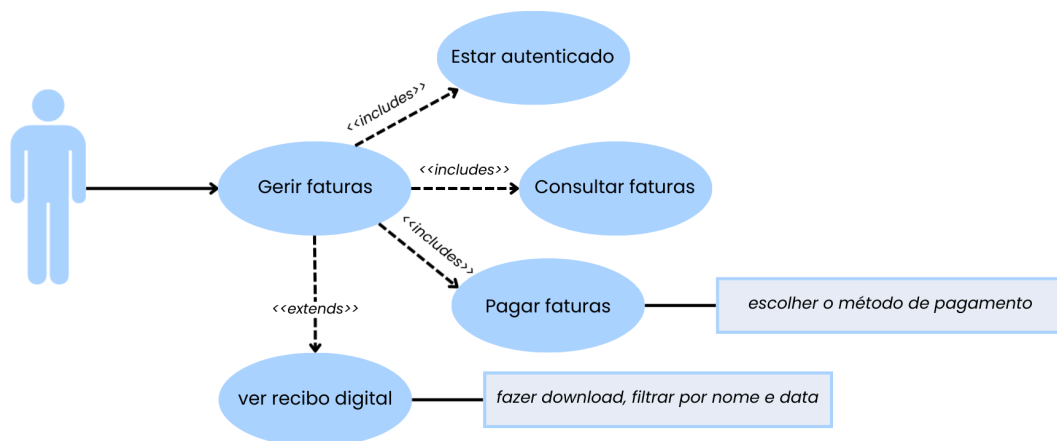


Diagrama de Sequência

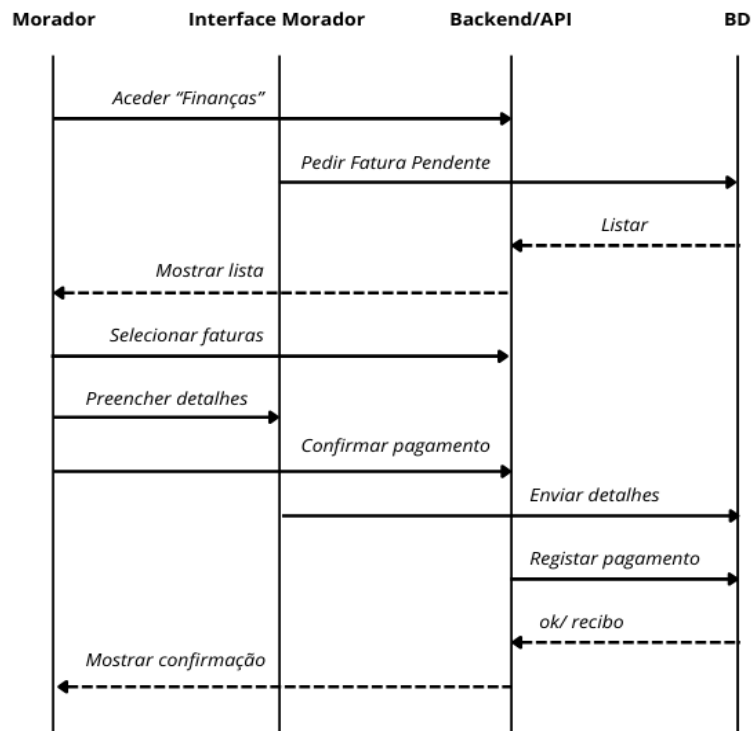
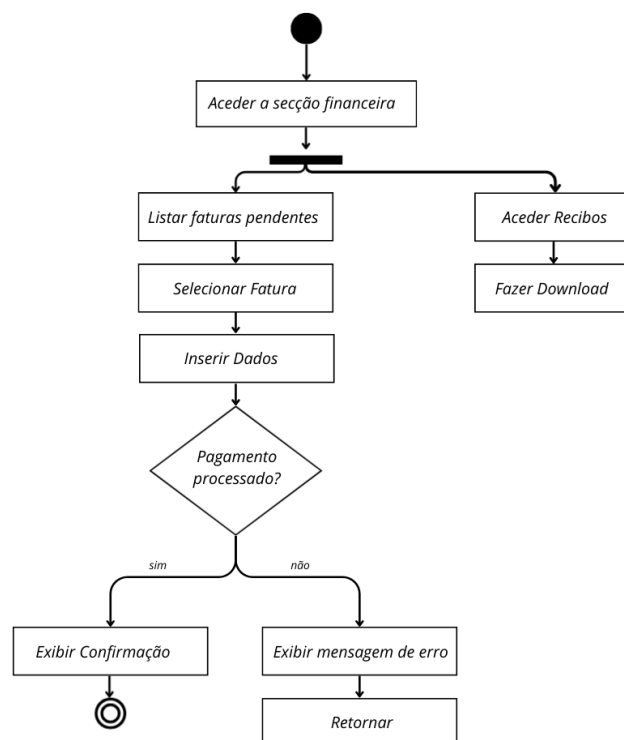


Diagrama de Atividades



Arquitetura da Solução e Decisões Tecnológicas

A arquitetura do MyKondô foi definida com base nos requisitos identificados e na necessidade de separar responsabilidades entre os diferentes componentes do sistema. A solução foi estruturada em aplicações cliente, serviços de backend e uma base de dados centralizada, permitindo uma comunicação clara entre camadas e facilitando a manutenção e evolução do sistema.

Para a aplicação mobile, foram utilizadas tecnologias orientadas ao desenvolvimento multiplataforma, nomeadamente **Expo** e **React Native**, permitindo a criação de uma aplicação acessível aos moradores com uma base de código comum. A utilização de **TypeScript** contribuiu para a robustez e manutenibilidade do código, reduzindo erros e facilitando a evolução da aplicação. Ferramentas adicionais, como **Expo Router**, **NativeWind** e **Expo Document Picker**, foram integradas para suportar navegação, estilização e funcionalidades específicas da aplicação.

A interface web administrativa foi desenvolvida com recurso a **Next.js**, **React** e **Tailwind CSS**, permitindo a criação de uma aplicação moderna, responsiva e orientada à gestão do condomínio. Esta abordagem facilitou a implementação de interfaces claras e organizadas, adequadas às necessidades do administrador.

O backend e a persistência de dados foram assegurados através do **Supabase**, com suporte de **PostgreSQL** como sistema de gestão de base de dados relacional. Esta escolha permitiu centralizar a informação, gerir autenticação e suportar comunicação em tempo real entre os diferentes componentes da plataforma. A comunicação entre frontend e backend é realizada através de **APIs REST/HTTP** e **Supabase Edge Functions**, garantindo flexibilidade e escalabilidade.

Para funcionalidades específicas, como o processamento de pagamentos, foi integrado o serviço **Stripe**, permitindo gerir transações de forma segura e transparente. No contexto da evolução da arquitetura e da exploração de conceitos de Sistemas Distribuídos, foi utilizada a ferramenta **Docker**, possibilitando a execução e gestão controlada dos serviços em ambiente local.

Inteligencia Artificial

Utilização de Problemas de Satisfação de Restrições (CSP) no Projeto

O projeto ao ser inserido no contexto da gestão de condomínios, tem como um dos principais objetivos apoiar os administradores nas decisões relacionadas com pedidos de manutenção efetuados pelos moradores. Estes pedidos podem variar em termos de tipo de serviço, urgência, orçamento e data pretendida, o que exige uma frequente seleção de um fornecedor externo capaz de executar o trabalho nas condições mais adequadas dadas.

Para responder a este desafio, o sistema integra um mecanismo de apoio à decisão baseado em **Problemas de Satisfação de Restrições (Constraint Satisfaction Problems – CSP)**. Esta abordagem permite modelar o problema de seleção de fornecedores de uma forma estruturada, garante que todas as decisões propostas respeitem um conjunto bem definido de regras e restrições do domínio.

Enquadramento do Problema

Quando um pedido é criado no sistema, o administrador necessita de identificar um fornecedor que:

- ofereça o serviço solicitado,
- esteja disponível na data e hora pretendidas,
- respeite, sempre que possível, o orçamento indicado
- possua uma avaliação adequada e histórico de desempenho positivo.

Este cenário envolve múltiplas entidades interligadas e diversas condições simultâneas, o que torna o problema adequado à modelação como um CSP, onde apenas combinações válidas e consistentes são consideradas soluções possíveis.

Modelação do CSP no Sistema

No sistema desenvolvido, o CSP é modelado através da definição explícita de **variáveis**, **domínios** e **restrições**, segue a estrutura clássica deste tipo de problema.

As variáveis principais do CSP são:

- **Fornecedor (X1)**: representa o prestador de serviços disponível no sistema;
- **Serviço (X)**: representa o tipo de serviço oferecido pelo fornecedor;
- **Slot de Tempo (X3)**: representa um intervalo temporal disponível na agenda do fornecedor.

Assim cada solução do CSP corresponde a uma combinação válida do tipo: (Fornecedor, Serviço, Slot de Tempo)

Figura 1 – Definição das variáveis do Problema de Satisfação de Restrições

```
const fornecedor = fornecedores[idx];
const servs = servicosPorFornecedor.get(fornecedor.id) ?? [];
const slotsF = slotsPorFornecedor.get(fornecedor.id) ?? [];
```

Figura 1 - Definição das variáveis CSP, onde o fornecedor, o serviço e o slot temporal constituem as variáveis principais do modelo.

Domínios das Variáveis

Os domínios das variáveis não são fixos, foram construídos dinamicamente com base nos dados que existem no sistema:

- O domínio da variável **Fornecedor**: inclui apenas fornecedores ativos e marcados como disponíveis;
- O domínio da variável **Serviço**: inclui apenas os serviços compatíveis com a categoria do pedido;
- O domínio da variável **Slot de Tempo**: inclui apenas os intervalos livres existentes na agenda de cada fornecedor.

Esta abordagem permite que o CSP se adapte automaticamente a alterações no sistema, como novos fornecedores, serviços ou horários.

Figura 2 – Construção dinâmica e indexação dos domínios

```
const servicosPorFornecedor = new Map<string, Servico[]>();
for (const s of servicos ?? []) {
  if (!servicosPorFornecedor.has(s.fornecedor_id)) {
    servicosPorFornecedor.set(s.fornecedor_id, []);
  }
  servicosPorFornecedor.get(s.fornecedor_id)!.push(s);
}

const slotsPorFornecedor = new Map<string, Slot[]>();
for (const z of slots ?? []) {
  if (!slotsPorFornecedor.has(z.fornecedor_id)) {
    slotsPorFornecedor.set(z.fornecedor_id, []);
  }
  slotsPorFornecedor.get(z.fornecedor_id)!.push(z);
}
```

Figura 2 – Construção e indexação dinâmica dos domínios do CSP, associando serviços e slots temporais a cada fornecedor disponível no sistema.

Restrições do Problema

Sobre as variáveis são aplicadas várias restrições, as que garantem que apenas soluções viáveis são consideradas. Entre as principais restrições implementadas destacam-se:

- **Restrição de compatibilidade de serviço:** o serviço selecionado deve corresponder à categoria do pedido;
- **Restrição de disponibilidade temporal:** o slot de tempo deve intersectar a data e hora pretendidas no pedido, considerando uma duração padrão do pedido de 60 minutos, assim simplifica a resolução do CSP e pode ser facilmente generalizada para durações variáveis;
- **Restrição de coerência fornecedor–slot:** o slot deve pertencer ao fornecedor selecionado;
- **Restrição de orçamento:** o preço médio do serviço não deve ultrapassar o orçamento máximo definido, sempre que aplicável;
- **Restrição de estado:** apenas slots com estado “livre” podem ser utilizados.

Qualquer combinação que viole uma destas restrições é automaticamente descartada durante o processo de resolução.

Figura 3 – Aplicação das restrições do CSP

```
for (const servico of servs) {  
  if (  
    pedido.orcamento_max != null &&  
    servico.preco_medio != null &&  
    servico.preco_medio > pedido.orcamento_max  
  ) {  
    continue;  
  }  
}
```

```
const intersecta =  
  slotInicio < pedidoFim &&  
  slotFim > pedidoInicio;  
  
if (!intersecta) continue;
```

Figura 3 – Aplicação das restrições do Problema de Satisfação de Restrições, incluindo validação de orçamento, verificação de interseção temporal e exclusão automática de combinações inválidas.

Processo de Resolução do CSP e Soluções

A resolução do Problema de Satisfação de Restrições é realizada no backend através de uma Edge Function dedicada. O algoritmo percorre de forma controlada as combinações possíveis entre fornecedores, serviços e slots temporais, aplicando todas as restrições definidas pelo domínio do problema.

Sempre que uma combinação viola alguma restrição, esta é imediatamente descartada, evitando a exploração de soluções inválidas. Este comportamento segue uma abordagem de **backtracking**, permitindo reduzir significativamente o espaço de pesquisa.

As soluções válidas encontradas correspondem a combinações consistentes do tipo (*Fornecedor, Serviço, Slot de Tempo*). O sistema limita o número máximo de soluções retornadas, garantindo um equilíbrio entre desempenho e utilidade prática.

As soluções são devolvidas à interface do administrador como **alternativas viáveis**, não sendo realizada qualquer escolha automática. A decisão final permanece sempre sob responsabilidade do utilizador (admin) que posteriormente solicita ao morador, o que reforça o papel do CSP como um **mecanismo de apoio à decisão** e não como um sistema de decisão autónomo.

Figura 4 – Algoritmo de resolução baseado em backtracking

```
function backtrackFornecedor(idx: number) {
  if (solutions.length >= max_solutions) return;
  if (idx >= fornecedores.length) return;

  const fornecedor = fornecedores[idx];
  const servs = servicosPorFornecedor.get(fornecedor.id) ?? [];
  const slotsF = slotsPorFornecedor.get(fornecedor.id) ?? [];

  for (const servico of servs) {
    if (
      pedido.orcamento_max != null &&
      servico.preco_medio != null &&
      servico.preco_medio > pedido.orcamento_max
    ) {
      continue;
    }

    for (const slot of slotsF) {
      const slotInicio = new Date(slot.inicio);
      const slotFim = new Date(slot.fim);
      const intersecta =
        slotInicio < pedidoFim &&
        slotFim > pedidoInicio;

      if (!intersecta) continue;
      const score =
        (fornecedor.avaliacao_media ?? 0) / 5;
    }
  }
}
```

```

        solutions.push({
            fornecedor_id: fornecedor.id,
            fornecedor_nome: fornecedor.nome,
            fornecedor_email: fornecedor.email,
            fornecedor_telefone: fornecedor.telefone,
            avaliacao_media: fornecedor.avaliacao_media,

            servico_id: servico.id,
            tipo_servico: servico.tipo_servico,
            preco_medio: servico.preco_medio,

            slot_id: slot.id,
            slot_inicio: slot.inicio,
            slot_fim: slot.fim,

            score,
        });

        if (solutions.length >= max_solutions) return;
    }
}

backtrackFornecedor(idx + 1);
}

backtrackFornecedor(0);

```

Figura 4 – Algoritmo de resolução do CSP baseado em backtracking, que explora combinações válidas de fornecedores, serviços e slots temporais, descartando antecipadamente soluções inválidas.

Integração no Sistema

As soluções resultantes do CSP são devolvidas à interface do sistema, onde o administrador pode analisá-las e selecionar a opção mais adequada. O sistema mantém sempre uma abordagem de **apoio à decisão**, não realizando a escolha final de forma automática.

Uma vez selecionada a solução, o pedido é aprovado e o slot correspondente é reservado, garantindo consistência entre a decisão tomada e o estado da agenda do fornecedor.

Aceder ao código através do repositório:

📁 back -> supabase_ha -> volumes -> function -> csp_match_fornecedores

Componentes de execução e integração das decisões (dentro da function)::

reservar_slot , fornecedor_upset_com_agenda , pedido_aprovar_com_slot

Sistemas Distribuídos

A arquitetura de sistema distribuído escolhida foi projetada para fornecer alta disponibilidade, continuidade de serviço e failover automático para a nossa plataforma, cobrindo os serviços de **Auth**, **RestAPI**, **Storage** e **Edge Functions**, bem como o banco de dados **PostgreSQL** subjacente. Para atender a esses requisitos, foi adotado o **Supabase em modo self-hosted**, que consiste na instalação e execução do Supabase localmente na infraestrutura própria (na máquina/servidor da aplicação), permitindo controle total sobre os componentes, configurações e fluxo de dados. Essa abordagem garante maior autonomia operacional, flexibilidade de customização e alinhamento com requisitos específicos de segurança e desempenho.

A solução foi concebida para minimizar indisponibilidades, reduzir riscos de inconsistência e permitir recuperação controlada em cenários de falha, mantendo decisões de roteamento sensíveis ao papel do banco de dados (**Primary** ou **Replica**). O uso do Supabase self-hosted possibilita a integração direta dos serviços com o PostgreSQL local, facilitando a implementação de estratégias de replicação, monitoramento e failover automático, além de assegurar a continuidade do serviço mesmo em situações de falha parcial da infraestrutura, preservando a integridade e a consistência dos dados.

Objetivos da arquitetura

Os principais objetivos do sistema são:

- Garantir alta disponibilidade dos serviços de API e banco de dados.
- Permitir failover automático do PostgreSQL, sem intervenção manual imediata
- Assegurar continuidade de serviço para clientes Web e Mobile
- Evitar cenários de split-brain e inconsistência grave de dados
- Prover observabilidade clara para operação, auditoria e troubleshooting
- Manter flexibilidade para evolução futura (ex.: leituras em réplica, automação de failback).

A arquitetura é organizada em duas camadas principais com redundância:

Camada de API

- Duas instâncias independentes do Supabase (Supabase 1 e Supabase 2)
- Cada instância expõe serviços via Kong (API Gateway).
- Balanceamento e decisão de roteamento realizados por HAProxy.

Camada de Banco de Dados

- PostgreSQL Primary e PostgreSQL Replica.
- Replicação assíncrona unidirecional.
- Failover automático com promoção da réplica
- Rejoin automático do antigo primário como réplica.

Essa separação permite escalar, monitorar e falhar componentes de forma independente, mantendo coerência operacional.

Componentes

Clientes

- Aplicações Mobile e Web
- Acedem a plataforma exclusivamente pela camada de API
- Não possuem conhecimento direto da topologia interna ou do papel do banco

Camada de API

1. Kong (API Gateway)

- Executado em duas instâncias independentes
- Responsável por autenticação, roteamento e controle de tráfego
- Mantém upstreams redundantes para serviços críticos, como Auth e o RestAPI

2. Serviços Supabase

- Auth
- PostgREST (RestAPI)
- Realtime (Websocket)
- Outros serviços auxiliares
-

Todos os serviços são geridos pelo Kong, garantindo um ponto único de entrada.

Balanceador de API

HAProxy (L4/L7)

- Decide para qual instância do Kong encaminhar o tráfego
- Pode priorizar o nó cuja instância de Postgres local seja Primary
- Essa estratégia reduz latência entre API e banco e melhora desempenho geral.

Camada de Banco de Dados

1. PostgreSQL Primary

- Responsável por todas as operações de escrita.
- Ponto de origem da replicação.

2. PostgreSQL Replica

- Recebe replicação assíncrona do primário.
- Pode ser promovida automaticamente em caso de falha.

Balanceador de Banco de Dados

HAProxy para PostgreSQL

- Separa tráfego de escrita e leitura
- Escritas sempre direcionadas ao Primary
- Leituras podem ser direcionadas ao Primary ou à Replica, dependendo da configuração.
- Atua como ponto único para failover transparente aos serviços

. Automação de Failover

1. Failover Watcher

- Monitora a disponibilidade do Primary.
- Promove automaticamente a Replica quando detecta falha
- Garante rápida retomada de operações de escrita.

2. Rejoin Automático

- Quando o antigo primário retorna, ele não reassume automaticamente.
- É reconfigurado como réplica, evitando conflitos e split-brain.

Fluxo de Tráfego

1. Tráfego da API

- Clientes enviam requisições para o HAProxy de API
- O HAProxy seleciona uma instância do Kong disponível
- O Kong encaminha a requisição para os serviços Supabase apropriados.

2. Tráfego de Banco de Dados

- Serviços Supabase se conectam ao HAProxy de banco.
- As operações de escrita são encaminhadas ao Primary.
- As operações de leitura podem ser direcionadas ao Primary ou à Réplica.
- A replicação ocorre do Primary para a Réplica de forma contínua.

Determinação de papéis do Banco e roteamento

O sistema utiliza mecanismos nativos do PostgreSQL para identificar o papel de cada nó:

- Primary: `pg_is_in_recovery () = false`
- Replica: `pg_is_in_recovery () = true`

Essas informações são expostas a agentes monitorados pelo HAProxy, permitindo:

- Atualização dinâmica dos backends ativos.
- Redirecionamento automático de tráfego após promoção de um nó.
- Decisões de roteamento consistentes com o estado real do banco.

Estratégias de Failover e Failback

1. Failover Automático

Ao detectar falha do Primary:

- A Réplica é promovida automaticamente
- O HAProxy passa a direcionar escritas para o novo Primary.
- Serviços Supabase continuam operando sem reconfiguração manual.

2. Failback Controlado

O antigo primário, ao retornar:

- Não reassume automaticamente o papel de Primary.
- É convertido em Replica após validação.
- Essa abordagem reduz drasticamente o risco de split-brain.

O failback é preferencialmente manual, garantindo controle operacional.

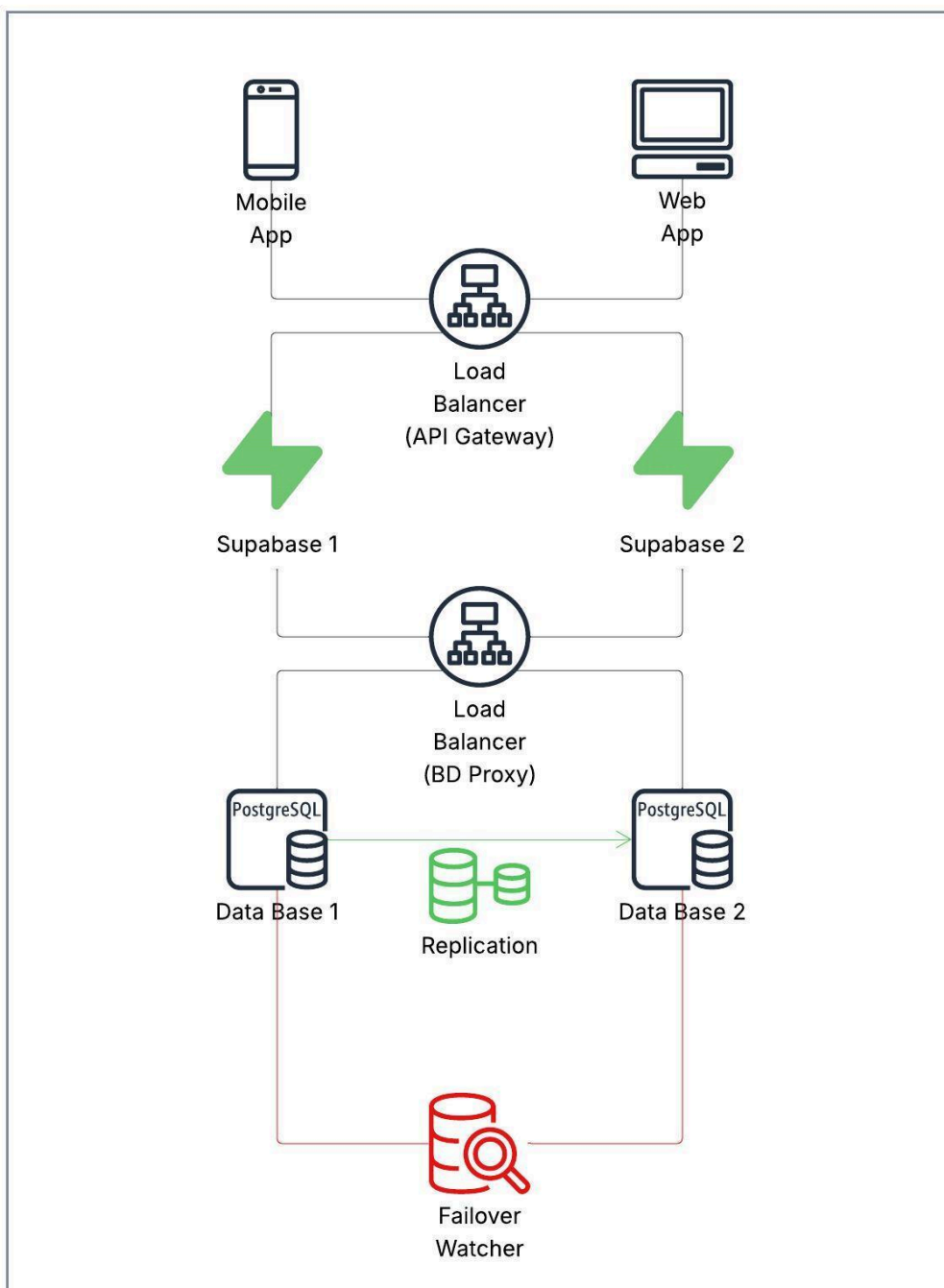
Consistência de dados

A replicação é assíncrona, o que implica:

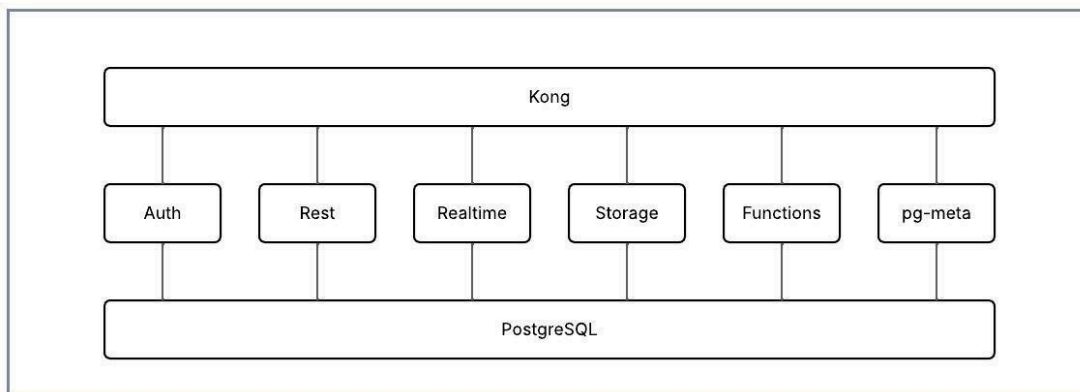
- **Pequena janela potencial de perda de dados em falhas severas**
- **Mitigações adotadas:**
 - Monitoramento de atraso de replicação.
 - Condicionamento da promoção a níveis aceitáveis de defasagem.
 - Disciplina rigorosa de migrações de esquema.

A consistência de esquema é crítica, pois divergências podem causar erros sistêmicos nos serviços de API.

Representação gráfica da Arquitetura



Estrutura Interna do Supabase



Implementações futuras

A arquitetura foi concebida com foco em extensibilidade, permitindo a incorporação de melhorias sem alterações estruturais profundas. Entre as principais evoluções previstas destacam-se:

Escalonamento de Leituras

Ativação controlada de leitura em réplicas para workloads intensivos, reduzindo carga no nó primário e melhorando latência para operações de consulta.

Observabilidade Centralizada

Consolidação de métricas, logs e alertas de HAProxy, Kong e PostgreSQL em dashboards unificados, facilitando análise de capacidade, detecção de anomalias e resposta a incidentes.

Automação Assistida de Failback

Introdução de playbooks semi-automatizados para retorno de primários, combinando validações técnicas com checkpoints humanos para minimizar riscos operacionais.

Testes de Resiliência (Chaos Testing)

Execução periódica de cenários controlados de falha para validar mecanismos de failover, tempos de recuperação e aderência aos SLOs definidos.

Aprimoramento de Segurança

Expansão do uso de TLS interno, rotação automatizada de segredos e políticas mais granulares de firewall entre camadas. Essas implementações reforçam a robustez da plataforma e garantem que a solução permaneça alinhada a requisitos crescentes de disponibilidade, desempenho e governança.

Cibersegurança

Importância da Segurança no Contexto do Projeto

A plataforma MyKondô lida com informação sensível de natureza pessoal, administrativa e financeira, incluindo dados de moradores, pagamentos, pedidos de manutenção e comunicações internas do condomínio. Neste contexto, a segurança da informação assume um papel central, sendo fundamental garantir a confidencialidade, integridade e disponibilidade dos dados armazenados e processados pelo sistema.

Para além da proteção contra acessos não autorizados, a solução deve assegurar que cada utilizador apenas tem acesso à informação correspondente ao seu perfil e às suas permissões, evitando fugas de dados e garantindo a confiança dos utilizadores na plataforma.

Autenticação e Autorização

A autenticação dos utilizadores é assegurada através dos mecanismos disponibilizados pelo Supabase, recorrendo a **tokens JWT (JSON Web Tokens)** para identificação segura das sessões. Este mecanismo permite validar a identidade dos utilizadores de forma eficiente, reduzindo o risco de acessos indevidos.

No que respeita à autorização, o sistema implementa um controlo rigoroso de acessos com base no perfil do utilizador, distinguindo claramente entre moradores e administradores. Esta separação garante que cada utilizador apenas pode executar operações compatíveis com o seu papel no sistema, como consultar faturas próprias ou gerir pedidos de manutenção a nível global.

O controlo de permissões é reforçado através da utilização de **Row Level Security (RLS)** ao nível da base de dados PostgreSQL. Estas políticas permitem definir regras de acesso diretamente sobre os dados, assegurando que mesmo tentativas de acesso fora da lógica da aplicação são bloqueadas pela própria base de dados.

Segurança das Comunicações

Todas as comunicações entre as aplicações cliente (mobile e web) e o backend são realizadas através de **ligações seguras**, utilizando protocolos HTTPS/TLS. Esta abordagem garante a proteção dos dados em trânsito, prevenindo ataques de interceção e garantindo a confidencialidade da informação transmitida.

A utilização de APIs REST e Edge Functions segue igualmente este princípio, assegurando que todas as interações entre componentes do sistema são protegidas contra acessos não autorizados ou manipulação de dados durante a transmissão.

Proteção e Gestão de Dados

A proteção dos dados armazenados é assegurada através de mecanismos de encriptação em repouso fornecidos pela infraestrutura subjacente ao Supabase e ao PostgreSQL. Esta abordagem reduz o impacto de eventuais acessos indevidos ao armazenamento físico dos dados.

Adicionalmente, o sistema recorre ao princípio do **menor privilégio**, utilizando diferentes chaves de acesso com níveis de permissões distintos, como chaves públicas de acesso restrito e chaves de serviço com permissões alargadas, utilizadas exclusivamente em contextos controlados do backend.

A existência de mecanismos de logging e monitorização contribui ainda para a deteção de comportamentos anómalos, auditoria de acessos e apoio à identificação de incidentes de segurança.

Boas Práticas de Desenvolvimento Seguro

Ao longo do desenvolvimento do projeto, foram adotadas boas práticas de programação segura, incluindo a validação de dados de entrada, a separação clara de responsabilidades entre componentes e a minimização da exposição de informação sensível nas interfaces cliente.

A utilização de tecnologias modernas e amplamente suportadas contribuiu igualmente para a redução de vulnerabilidades conhecidas, beneficiando de atualizações regulares e de uma comunidade ativa de desenvolvimento.

Limitações e Evoluções Futuras

Apesar dos mecanismos de segurança implementados responderem aos requisitos atuais do projeto, existem oportunidades de evolução que poderão aumentar o nível de maturidade da solução. Entre as melhorias futuras destaca-se a introdução de **autenticação multifator (MFA)**, reforçando a proteção das contas de utilizador.

Outras evoluções possíveis incluem o reforço das políticas de auditoria e logging, a rotação automática de credenciais e a implementação de controlos de acesso ainda mais granulares. Estas melhorias permitiriam aumentar a robustez da plataforma face a ameaças emergentes e preparar o sistema para cenários de maior escala e exigência.

Conclusão

O projeto **MyKondô** teve como objetivo o desenvolvimento de uma plataforma digital integrada para a gestão de condomínios, procurando responder às limitações identificadas nos processos tradicionais de administração e comunicação entre moradores e gestores. Ao longo do projeto, foi possível conceber e implementar uma solução funcional que centraliza informação, automatiza processos e promove uma interação mais eficiente entre os diferentes intervenientes.

A abordagem adotada permitiu transformar um problema real num sistema coerente, suportado por decisões técnicas fundamentadas e por uma arquitetura preparada para evolução. A aplicação dos princípios da Engenharia de Software foi essencial para estruturar o desenvolvimento do projeto, desde a análise de requisitos até à modelação do sistema, garantindo alinhamento entre as necessidades dos utilizadores e a solução implementada.

A integração de uma componente de **Inteligência Artificial**, baseada em Problemas de Satisfação de Restrições, revelou-se eficaz no apoio à tomada de decisão da administração, permitindo recomendar fornecedores de forma automática e transparente. Esta funcionalidade contribui para a redução do esforço manual, melhoria dos tempos de resposta e aumento da qualidade do serviço prestado aos moradores.

Do ponto de vista da infraestrutura, a evolução para uma arquitetura de **Sistemas Distribuídos** permitiu explorar mecanismos de disponibilidade, tolerância a falhas e consistência, reforçando a robustez da solução. A adaptação do Supabase para execução em ambiente local e a implementação de mecanismos de replicação e failover evidenciam a capacidade de adaptação da arquitetura face aos desafios identificados durante o desenvolvimento.

A **Cibersegurança** assumiu igualmente um papel central, sendo adotados mecanismos de autenticação, autorização, controlo de acessos e proteção de dados, garantindo a confidencialidade, integridade e disponibilidade da informação. Estas medidas contribuem para a confiança dos utilizadores na plataforma e para a sustentabilidade da solução a longo prazo.

Em suma, o MyKondô constitui uma solução integrada e funcional que responde aos objetivos inicialmente definidos, demonstrando a aplicação prática de conceitos fundamentais nas áreas de Engenharia de Software, Inteligência Artificial, Sistemas Distribuídos e Segurança

Reflexão Crítica e Trabalho Futuro

O desenvolvimento do projeto MyKondô permitiu consolidar conhecimentos teóricos através da sua aplicação prática num contexto realista e tecnicamente exigente. Ao longo do projeto, foram enfrentados diversos desafios, desde a definição inicial do problema até à adaptação da arquitetura face a limitações técnicas identificadas durante a implementação.

Um dos principais aspetos positivos do projeto foi a capacidade de evolução e adaptação da solução. A necessidade de reformular decisões iniciais, nomeadamente ao nível da arquitetura distribuída, evidenciou a importância de uma abordagem iterativa e flexível no desenvolvimento de sistemas complexos. Esta experiência permitiu compreender melhor o impacto das decisões técnicas e a importância de avaliar continuamente a adequação das soluções adotadas.

Apesar dos resultados alcançados, o projeto apresenta limitações que poderão ser endereçadas em trabalhos futuros. A componente de Inteligência Artificial poderá ser expandida com mecanismos de aprendizagem baseados em dados históricos, permitindo recomendações mais personalizadas e adaptativas. Ao nível da infraestrutura, poderão ser exploradas estratégias mais avançadas de escalabilidade, observabilidade e testes de resiliência, reforçando a robustez da solução em cenários de maior carga.

Adicionalmente, a plataforma poderá evoluir em termos de funcionalidades, integrando novos serviços e melhorando a experiência do utilizador, tanto para moradores como para administradores. Estas evoluções permitiriam aumentar o impacto prático da solução e aproximá-la ainda mais de um sistema utilizável em contexto real.

Em conclusão, o projeto MyKondô constituiu uma experiência enriquecedora, permitindo o desenvolvimento de competências técnicas e metodológicas relevantes, bem como uma compreensão mais aprofundada dos desafios associados à conceção e implementação de sistemas de software complexos.

Anexos

Link para ter acesso ao repositório, GitHub:

https://github.com/AndyMorais1/projeto_mobile_2025_2026/tree/dev